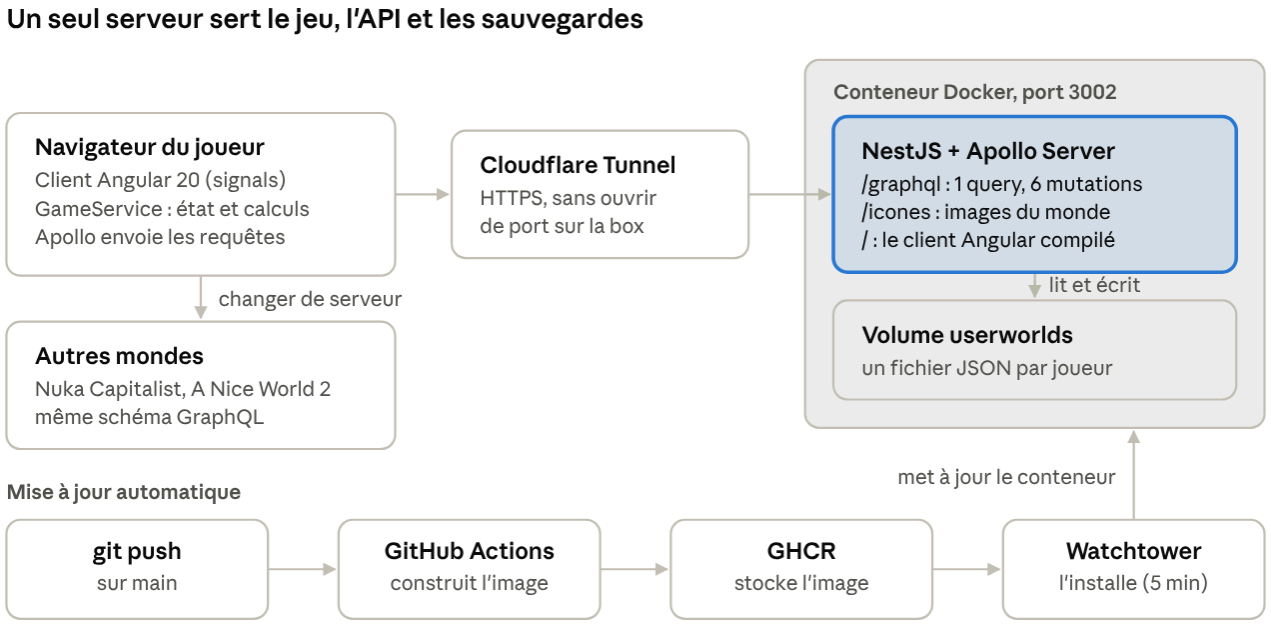
ISIS Capitalist — mémo technique

Oct 19, 2026 · @Alexis DECHENAIS

Adventure Time Capitalist est un clone d'Adventure Capitalist : un serveur NestJS expose le monde de chaque joueur en GraphQL, un client Angular 20 l'affiche et l'anime, et l'ensemble tourne dans un conteneur Docker sur capitalist.alexisdechenais.fr. Ce mémo résume tout ce qui a été fait, pour s'en souvenir avant la soutenance.

Vue d'ensemble



architecture · client, serveur, sauvegardes, déploiement

Le navigateur parle au serveur en GraphQL, à travers le tunnel Cloudflare ; le serveur lit et écrit la sauvegarde du joueur à chaque requête. Le même client peut aussi viser le serveur d'un autre groupe.

Les technologies

Tout est écrit en TypeScript, des deux côtés.

Outil	Version	Côté	À quoi il sert
Node.js	24	Les deux	Exécute le serveur et les outils de build

Outil	Version	Côté	À quoi il sert
NestJS	12	Serveur	Framework serveur : modules, services, injection de dépendances
GraphQL + @nestjs/graphql	16 / 14	Serveur	Le protocole de l'API : un schéma typé, des queries (lecture) et des mutations (écriture)
Apollo Server	5	Serveur	Le moteur qui répond aux requêtes GraphQL sur `/graphql`
ts-morph	24	Serveur	Génère `graphql.ts` (les classes TypeScript) à partir du schéma
@nestjs/serve-static	12	Serveur	Sert les images du monde (`/icones`) et, en production, le client Angular
Angular	20	Client	Framework du client : composants autonomes (standalone) et signals
Angular Material	20	Client	Le SnackBar (messages temporaires)
apollo-angular + @apollo/client	14 / 4	Client	Envoie les queries et mutations GraphQL au serveur
graphql-codegen	7	Client	Génère les types et les requêtes TypeScript depuis le schéma (`schema-types.ts` , `operations.ts`)
RxJS	7	Les deux	Flux asynchrones ; on les convertit en promesses avec `firstValueFrom`
Vitest	4	Serveur	Tests automatisés du serveur
Karma + Jasmine	6 / 5	Client	Tests automatisés du client, dans Chrome sans fenêtre
Docker	—	Déploiement	Une image unique qui contient le serveur et le client
GitHub Actions + GHCR	—	Déploiement	Construit l'image à chaque push sur `main` et la publie
Watchtower	—	Déploiement	Sur notre serveur, installe la nouvelle image automatiquement
Cloudflare Tunnel	—	Déploiement	Publie le jeu en HTTPS sans ouvrir de port sur la box

Le backend (dossier `backend/`)

Le serveur est la source de vérité : chaque requête relit le monde du joueur, le fait avancer jusqu'à maintenant, applique l'action, puis le sauvegarde.

Les fichiers

Fichier	Rôle
`src/schema.graphql`	Le contrat : types `World`, `Product`, `Palier`, l'enum `RatioType` (gain, vitesse, ange), la query et les mutations
`src/resolver.ts`	Reçoit chaque query ou mutation : lit le monde, appelle le service, sauvegarde, répond
`src/app.service.ts`	Toute la logique du jeu : achats, managers, upgrades, unlocks, anges, reset, lecture et écriture des sauvegardes
`src/production.ts`	`calcQtProductionforElapseTime` : combien de productions se sont terminées pendant un temps donné
`src/origworld.ts`	Le monde de départ (6 produits, 6 managers, unlocks, upgrades)
`src/app.module.ts`	Branche GraphQL et le service des fichiers statiques
`userworlds/`	Une sauvegarde JSON par joueur : `<pseudo>-world.json`

L'API GraphQL

Opération	Type	Ce qu'elle fait
`getWorld(user)`	Query	Renvoie le monde du joueur, mis à jour jusqu'à maintenant (créé depuis `origworld` s'il n'existe pas)
`acheterQtProduit(user, id, quantite)`	Mutation	Achète n exemplaires : débite le coût cumulé, augmente la quantité, met à jour le prix du suivant, débloque les paliers atteints

Opération	Type	Ce qu'elle fait
`lancerProductionProduit(user, id)`	Mutation	Démarre une production : `timeleft = vitesse`
`engagerManager(user, name)`	Mutation	Paie le manager et passe `managerUnlocked` à vrai sur son produit
`acheterCashUpgrade(user, name)`	Mutation	Paie en argent et applique le bonus
`acheterAngelUpgrade(user, name)`	Mutation	Paie en anges et applique le bonus
`resetWorld(user)`	Mutation	Repart du monde d'origine en gardant score et anges

Le temps qui passe

- `lastupdate` (en millisecondes, stocké en texte) retient le dernier passage.
- À chaque lecture, `evolveWorld` calcule le temps écoulé et, pour chaque produit, le nombre de productions terminées via `calcQtProductionforElapseTime`. Il crédite `qt × revenu × quantité × (1 + anges actifs × bonus / 100)` à l'argent et au score.
- Sans manager, un produit fait au plus 1 production puis s'arrête (`timeleft = 0`). Avec manager, il enchaîne les cycles et garde le reste pour le cycle suivant.
- La sauvegarde est **synchrone** (`writeFileSync`) : une écriture asynchrone pouvait perdre des mises à jour quand les requêtes s'enchaînent.

Paliers et bonus

- Chaque produit a 3 paliers de quantité (20, 75, 300) qui divisent sa vitesse.
- Les « allunlocks » se débloquent quand **tous** les produits atteignent 30, 150 ou 1000 exemplaires, et multiplient les gains de tous les produits.
- Un bonus `gain` multiplie le revenu, un bonus `vitesse` divise la vitesse (arrondie à l'entier), un bonus `ange` **ajoute** son ratio au bonus par ange (2 % devient 4 %).

Anges et reset

- Anges gagnés = 150 × √(score / 10^15), arrondi à l'entier inférieur : il faut un score de 4,44 × 10^10 pour le premier.
- `totalangels` est tenu à jour par le serveur à chaque lecture et ne redescend jamais.
- Au reset : on repart de `origworld` , on garde le score, anges actifs = total − anges dépensés en angel upgrades. Les angel upgrades déjà achetés restent acquis et leur bonus est réappliqué.

Sécurité

Le pseudo devient un nom de fichier : il est encodé (`encodeURIComponent`) et limité à 64 caractères, pour qu'un pseudo comme `../../x` ne puisse pas écrire hors du dossier des sauvegardes.

Le frontend (dossier `frontend/`)

Le client met à jour son propre état tout de suite (affichage fluide), puis envoie l'action au serveur ; il ne recharge le monde entier qu'aux moments prévus.

Les fichiers

Fichier	Rôle
`app/game.service.ts`	Le cœur du client : le signal `world` , le pseudo, le multiplicateur, les achats, managers, upgrades, reset, et tous les appels GraphQL
`app/app.ts` + `app.html`	La page : menu « Finn's Pack », bandeau du trésor, grille des produits, fenêtres (Managers, Unlocks, Upgrades, Angels, Prismo's Wishes), badges
`app/produit/`	Une carte produit : icône, barre de progression Jake, bouton d'achat, temps restant
`app/stats/`	La carte « Swag and Stats » de Finn (trésor, produits, vœux, temps de jeu)
`app/world-servers.ts`	La liste des mondes et l'adresse du serveur courant, gardées dans le localStorage
`app/sound.service.ts`	Les effets sonores (Lemongrab, BMO) et leur bouton on/off
`app/bigvalue-pipe.ts` , `second-pipe.ts`	Formatage des grands nombres (4 chiffres, puis notation 10^n) et des durées
`app/graphql/`	`queries.graphql` (nos requêtes) + `schema-types.ts` et `operations.ts` générés par codegen
`public/`	Pièce d'or, musique, sons : communs à tous les mondes

Signals et GraphQL

- `world`, `user`, `qtmulti` et l'adresse du serveur sont des **signals** : l'affichage se met à jour tout seul quand ils changent.
- Les badges, le revenu par seconde et le nombre d'anges à réclamer sont des `computed` : recalculés automatiquement à partir de `world`.
- Le sujet décrit `apollo.signal.query`, qui n'existe pas dans la version publiée d'apollo-angular : on appelle `Apollo.query` / `mutate`, converties en promesses, et on écrit le résultat dans le signal `world`.
- Le lien HTTP d'Apollo lit l'adresse du serveur à chaque requête : changer de monde ne demande pas de recréer le client. Les queries n'utilisent pas le cache (`no-cache`), car deux serveurs peuvent avoir des produits de même id.
- `getWorld` n'est appelé qu'au chargement, au Refresh, au reset, à l'ouverture d'Investors, et en changeant de joueur ou de serveur.

Achat et calculs côté client

- Multiplicateur : x1 → x10 → x100 → Max → x1.
- Coût de n exemplaires = coût × (1 + c + c² + … + c^(n−1)), c étant la croissance du produit.
- En Max : on ajoute des exemplaires tant que le coût cumulé reste inférieur à l'argent.
- `canBuy` active le bouton seulement si les fonds suffisent. Un clic sur un bouton grisé joue « UNACCEPTABLE! ».
- Les paliers et bonus sont appliqués côté client avec les mêmes règles que le serveur, pour un effet immédiat.

La barre de production

- Animée avec `requestAnimationFrame` : progression = temps écoulé / vitesse du produit.
- Avec un manager, la production redémarre seule ; si plusieurs cycles se terminent entre deux images, ils sont tous crédités.
- Sous 300 ms par cycle, la barre reste pleine (mode « continuous ») et affiche le gain par seconde, comme dans le vrai jeu.
- Après un rechargement du monde (`worldVersion`), les animations en cours sont abandonnées et reprennent là où le serveur en est.

Retours visuels et sonores

- SnackBar (Angular Material) pour chaque achat, manager, palier, reset et erreur serveur.

- Badges sur Managers, Cash Upgrades et Angel Upgrades quand un achat est possible.
- Musique de fond en boucle (démarre au premier clic, les navigateurs bloquant la lecture automatique) ; bips de BMO quand un palier se débloque.

Le monde Adventure Time

Six objets emblématiques de la série, dans l'ordre chronologique de l'histoire, chacun géré par un personnage. L'économie est volontairement rapide pour une démo courte.

Produits et managers

#	Produit	Coût de départ	Croissance	Revenu par cycle	Durée d'un cycle	Manager	Prix du manager
1	Ice King's Crown	4	×1,07	200	0,5 s	Simon Petrikov	2 400
2	The Enchiridion	60	×1,10	12 000	3 s	Simon the Antiquarian	960 000
3	Marceline's Axe Bass	720	×1,12	108 000	6 s	Marceline Abadeer	7,2 M
4	Finn's Root Sword	8 640	×1,13	864 000	12 s	Finn the Human	48 M
5	The Nightosphere Gauntlet	103 680	×1,14	10,4 M	24 s	Billy the Hero	360 M
6	Finn's Golden Sword of Battle	1,24 M	×1,15	124 M	48 s	Princess Bubblegum	3,6 Md

Chaque produit a trois paliers de quantité : 20 (vitesse ÷2), 75 (÷2) et 300 (÷3).

Bonus globaux

Nom	Type	Condition	Effet
The Great Mushroom War ends	Unlock	30 de chaque produit	Gains de tout ×2
The Cosmic Owl's prophecy	Unlock	150 de chaque produit	Gains de tout ×3
GOLB awakens at the edge of Ooo	Unlock	1 000 de chaque produit	Gains de tout ×4
Prismo's wish	Angel upgrade	1 ange	Gains de tout ×3

Nom	Type	Condition	Effet
Death's bargain	Angel upgrade	4 anges	Gains de tout ×4
The Cosmic Owl's dream	Angel upgrade	10 anges	+2 % par ange

Les 10 Cash Upgrades multiplient les gains d'un produit précis par 3 ou 4, pour 80 000 à 320 M d'argent.

L'habillage

- Jake sert de barre de progression : il s'étire quand la production avance.
- Ciel et herbe en fond, contours noirs épais, polices Chewy et Grandstander.
- Le menu est le sac de Finn (« Finn's Pack »), les Angel Investors deviennent « Prismo's Wishes ».
- Sons : Lemongrab crie « UNACCEPTABLE! » quand on ne peut pas acheter, BMO bipe quand un palier se débloque.

Interopérabilité

Tous les groupes partagent le même schéma GraphQL : notre client peut jouer sur le serveur d'un autre groupe, et inversement.

- La liste des mondes est en haut à droite. Par défaut : notre monde et Nuka Capitalist (`https://capitalist.lucaslopvet.fr`).
- **+** ajoute un monde à partir de son adresse. Il n'est gardé que si son serveur renvoie bien un monde ; sinon un message d'erreur s'affiche et on revient au monde précédent.
- **−** retire le monde affiché (il en reste toujours au moins un).
- L'adresse est nettoyée : `/graphql` final retiré, et `http://` ou `https://` ajouté s'il manque.
- Les images d'un monde viennent de son propre serveur (`<adresse>/icones/...`) ; la pièce, la musique et les sons viennent de notre client, communs à tous les mondes.
- Liste des mondes et serveur choisi sont gardés dans le localStorage. En ligne, le monde par défaut est l'adresse du site lui-même.
- Pour qu'un autre groupe ajoute notre monde : `https://capitalist.alexisdechenais.fr/graphql`. Le CORS est activé côté serveur pour l'autoriser.

Tests automatisés

72 tests au total, tous au vert : 51 côté serveur et 21 côté client.

Côté	Outil	Commande	Ce qui est testé
Serveur	Vitest	`npm test` dans `backend/`	Les 21 cas officiels de `calcQtProductionforElapseTime` ; achat (coût cumulé, argent insuffisant) ; paliers et allunlocks ; managers ; upgrades et angel upgrades ; reset et anges ; vitesse gardée entière après un bonus
Client	Karma + Jasmine	`npx ng test --watch=false` dans `frontend/`	Le GameService (coûts, quantité en mode Max, achat, managers, unlocks, upgrades) ; le nettoyage des adresses de monde ; la construction des adresses d'images ; le composant principal

Les tests ne remplacent pas la vérification à la main : chaque ligne de la grille a aussi été testée dans le navigateur.

Mise en ligne

Le jeu tourne sur le serveur maison d'Alexis (mini PC Ubuntu) et se met à jour tout seul à chaque `git push` sur `main`.

1. **Une seule image Docker** (`Dockerfile`) : elle compile le client Angular et le serveur NestJS, puis le serveur sert les deux sur un seul port. Une seule adresse pour jouer et pour l'API.
2. **GitHub Actions** (`.github/workflows/docker.yml`) construit l'image à chaque push sur `main` et la publie sur GHCR : `ghcr.io/alexisdcn/isiscapitalist:latest`.
3. **Dockge** fait tourner la stack (`deploy/compose.yaml`) sur le port 3002. Les sauvegardes sont dans un volume Docker, conservé entre les mises à jour.
4. **Watchtower** vérifie toutes les 5 minutes s'il y a une nouvelle image et l'installe.
5. **Cloudflare Tunnel** publie le jeu en HTTPS sur `capitalist.alexisdechenais.fr`, sans ouvrir de port sur la box. `/graphql` reste public pour que les autres groupes puissent nous rejoindre.

Avant le build, un script (`generate-typings.mjs`) génère `graphql.ts` à partir du schéma, car ce fichier n'est pas versionné. Le guide pas à pas est dans `deploy/README.md` .

Problèmes rencontrés et solutions

De bons exemples à raconter si le jury demande une difficulté.

Problème	Cause	Solution
`apollo.signal.query` du sujet introuvable	Cette API n'existe pas dans le paquet apollo-angular publié	Un signal écrit à la main, rempli par `Apollo.query` / `mutate`
Des achats perdus côté serveur	La sauvegarde asynchrone n'était pas finie quand la requête suivante relisait le fichier	Sauvegarde synchrone (`writeFileSync`)
Mon monde ne se chargeait plus	Un bonus de vitesse donnait 41,67 ms alors que `vitesse` est un entier dans le schéma	Arrondi de la vitesse, et réparation des anciennes sauvegardes à la lecture
Argent compté en trop après un reset	Des animations de l'ancien monde continuaient de créditer des gains	Compteur `worldVersion` : un rechargement annule les animations en cours
Barre qui clignotait à très haute vitesse	Impossible d'afficher un remplissage lisible sous 300 ms	Mode « continuous » : barre pleine et gain par seconde
Impossible d'avoir des anges en démo	Le premier ange demande un score de 4,44 × 10^10	Économie accélérée pour une partie courte
Premier build Docker en échec	Le lockfile du client, généré sous Windows, n'avait pas les versions Linux de `@parcel/watcher`	Lockfile régénéré avec la même version de npm que GitHub
Faille de sécurité avant la mise en ligne	Le pseudo devenait un nom de fichier tel quel	Pseudo encodé et limité à 64 caractères

Glossaire

Mot	En une phrase
Query	Une requête GraphQL qui lit des données sans rien modifier (`getWorld`).
Mutation	Une requête GraphQL qui modifie des données (acheter, engager, reset).
Resolver	La fonction serveur qui répond à une query ou une mutation.
Schema first	On écrit le schéma GraphQL d'abord, le code TypeScript en est généré.
Codegen	L'outil qui génère automatiquement types et requêtes TypeScript à partir du schéma.
Signal	Une valeur réactive d'Angular : tout ce qui la lit se met à jour quand elle change.
`computed`	Un signal calculé à partir d'autres signals (exemple : le badge Managers).
`effect`	Du code relancé automatiquement quand un signal qu'il lit change (exemple : afficher le SnackBar).
Composant standalone	Un composant Angular autonome, sans NgModule.
Injection de dépendances	Le framework fournit les services aux classes qui en ont besoin (`inject(GameService)`).
localStorage	Un petit stockage du navigateur qui survit au rechargement (pseudo, liste des mondes, son).
CORS	La règle qui autorise une page d'un autre site à appeler notre serveur.
`lastupdate`	L'instant de la dernière mise à jour du monde, pour calculer le temps écoulé.
Palier	Un objet bonus (unlock, upgrade, manager) : seuil, cible, ratio et type de bonus.
Image Docker	Un paquet qui contient l'application et tout ce qu'il lui faut pour tourner, partout pareil.
CI/CD	Construction et déploiement automatiques à chaque push (GitHub Actions + Watchtower).

Mot	En une phrase
Tunnel Cloudflare	Une connexion sortante du serveur vers Cloudflare, qui publie le site sans ouvrir de port.

Questions sur le code : backend

Chaque question montre le vrai code du projet, puis ce qu'il faut dire. Le principe à retenir : **lire le monde, le faire avancer, appliquer l'action, sauvegarder, répondre**.

« Que se passe-t-il quand le client appelle une mutation ? »

`backend/src/resolver.ts`

```
@Mutation()
async acheterQtProduit(
  @Args('user') user: string,
  @Args('id') id: number,
  @Args('quantite') quantite: number,
): Promise<Product> {
  const world = this.service.readUserWorld(user);
  const product = this.service.achatProduit(world, id, quantite);
  this.service.saveWorld(user, world);
  return product;
}
```

À dire : `@Mutation()` relie la méthode à la mutation du même nom dans `schema.graphql`, et `@Args` récupère ses arguments. Le resolver ne contient pas de logique : il lit le monde (qui est mis à jour au passage), délègue au service, sauvegarde et renvoie le produit. Toutes les mutations suivent ce même schéma.

« D'où vient `this.service` ? »

```
@Resolver('World')
export class GraphQlResolver {
  constructor(private service: AppService) {}
```

À dire : c'est l'**injection de dépendances** de NestJS. `AppService` est déclaré dans les `providers` de `app.module.ts` ; Nest crée une seule instance et la passe au constructeur. On ne fait jamais `new AppService()`.

« Comment l'argent évolue-t-il quand le joueur n'est pas là ? »

`backend/src/app.service.ts`, appelé à chaque lecture du monde

```
private evolveWorld(world: World): void {
  const now = Date.now();
  const elapseTime = Math.max(0, now - Number(world.lastupdate));
  if (elapseTime > 0) {
    for (const product of world.products) {
      const qt = calcQtProductionforElapseTime(product, elapseTime);
      if (qt > 0) {
        const gain = qt * this.calculerGainProduit(world, product);
        world.money += gain;
        world.score += gain;
      }
    }
  }
  world.totalangels = Math.max(world.totalangels,
this.angesGagnes(world.score));
  world.lastupdate = String(now);
}
```

À dire : le serveur ne tourne pas en boucle. Il calcule « après coup » : temps écoulé depuis `lastupdate`, nombre de productions terminées par produit, gain crédité à l'argent **et** au score (le score sert aux anges). Puis `lastupdate` passe à maintenant. `lastupdate` est un `String` dans le schéma parce qu'un nombre de millisecondes dépasse la taille d'un `Int` GraphQL.

« Expliquez `calcQtProductionforElapseTime`. »

`backend/src/production.ts`

```
if (product.timeleft === 0) return 0;          // rien ne tourne
if (elapseTime < product.timeleft) {           // pas encore fini
  product.timeleft -= elapseTime;
  return 0;
}
if (!product.managerUnlocked) {                // fini, et on s'arrête
  product.timeleft = 0;
  return 1;
}
const tempsRestant = elapseTime - product.timeleft;   // manager : on
enchaîne
const cyclesSupplementaires = Math.floor(tempsRestant / product.vitesse);
const reste = tempsRestant - cyclesSupplementaires * product.vitesse;
product.timeleft = product.vitesse - reste;
return 1 + cyclesSupplementaires;
```

À dire : quatre cas. Rien en cours : 0. Pas encore fini : on retire le temps écoulé. Fini sans manager : 1 production, puis arrêt. Avec manager : la production en cours, plus autant de cycles complets que le temps restant le permet, et le reste devient le début du cycle suivant. La fonction est testée sur les 21 cas officiels du sujet.

« Comment calculez-vous le coût d'un achat multiple ? »

```
let coutUnitaire = product.cout;
let coutTotal = 0;
for (let i = 0; i < quantite; i++) {
  coutTotal += coutUnitaire;
  coutUnitaire *= product.croissance;
}
if (world.money < coutTotal) throw new Error(`Not enough money ...`);
world.money -= coutTotal;
product.quantite += quantite;
product.cout = coutUnitaire;
```

À dire : `cout` est toujours le prix du **prochain** exemplaire. Chaque exemplaire coûte le précédent multiplié par la croissance ; on additionne, on vérifie l'argent, puis on enregistre le nouveau prix. C'est une suite géométrique : on aurait pu utiliser la formule `cout × (c^n − 1) / (c − 1)`, la boucle est plus lisible.

« Que se passe-t-il si le client envoie une action impossible ? »

À dire : le service lève une `Error` (pas assez d'argent, manager déjà engagé, produit inconnu). Apollo Server la transforme en réponse GraphQL avec un champ `errors`, et rien n'est sauvegardé puisque `saveWorld` n'est pas atteint. Côté client, cela affiche «

Action rejected by the server ». Le serveur ne fait jamais confiance au client.

« Comment se débloquent les paliers ? »

```
private checkAllUnlocks(world: World) {
  const quantiteMin = Math.min(...world.products.map((p) => p.quantite));
  for (const palier of world.allunlocks) {
    if (!palier.unlocked && quantiteMin >= palier.seuil) {
      palier.unlocked = true;
      this.applyPalierBonus(world, palier);
    }
  }
}
```

À dire : après chaque achat, on compare la quantité aux seuils. Pour un palier de produit, c'est la quantité de ce produit ; pour un allunlock, c'est la quantité du produit **le moins possédé** (d'où le `Math.min`). Un bonus n'est appliqué qu'une fois grâce à `unlocked`.

« Où et comment sont stockées les parties ? »

À dire : un fichier JSON par joueur, lu avec `readFileSync` et écrit avec `writeFileSync`. Si le fichier n'existe pas, on part d'une copie de `origworld` (avec `structuredClone`, pour ne jamais modifier le monde d'origine partagé par tous). L'écriture est synchrone pour qu'une requête ne relise jamais un fichier à moitié écrit.

« D'où viennent les classes `World`, `Product`, `Palier` ? »

À dire : elles ne sont pas écrites à la main. Avec l'option `definitions` de `GraphQLModule`, Nest génère `src/graphql.ts` à partir de `schema.graphql` au démarrage. C'est l'approche **schema first** : le schéma est la référence, le code en découle.

Questions sur le code : frontend

Le principe à retenir : **le signal `world` est la seule source de l'affichage**. On le modifie tout de suite pour la fluidité, et le serveur reçoit l'action en parallèle.

« Comment le client est-il relié au serveur GraphQL ? »

`frontend/src/app/app.config.ts`

```
provideApollo(() => {
  const httpLink = inject(HttpLink);
  return {
    link: httpLink.create({ uri: () => serverUrl() + '/graphql' }),
    cache: new InMemoryCache(),
  };
}),
```

À dire : Apollo est déclaré une fois pour toute l'application. L'adresse est une **fonction** qui lit le signal `serverUrl` à chaque requête : quand on change de monde, la requête suivante part vers le nouveau serveur sans rien recréer.

« Comment chargez-vous le monde ? »

`frontend/src/app/game.service.ts`

```
readonly world = signal<World | undefined>(undefined);

async refreshWorld(douce = false): Promise<boolean> {
  try {
    const { data } = await firstValueFrom(
      this.apollo.query({
        query: GetWorldDocument,
        variables: { user: this.user() },
        fetchPolicy: 'no-cache',
      }),
    );
    this.world.set(data.getWorld as World);
    return true;
  } catch {
    this.notify('Could not load the world from the server');
    return false;
  }
}
```

À dire : `GetWorldDocument` est généré par codegen à partir de `queries.graphql`, donc la réponse est typée. `apollo.query` renvoie un Observable ; `firstValueFrom` le transforme en promesse pour écrire un code linéaire avec `await`. Le résultat va dans le signal `world`, et tout l'affichage se met à jour. `no-cache` évite qu'Apollo mélange deux mondes dont les produits ont les mêmes id.

« Pourquoi pas `apollo.signal.query` comme dans le sujet ? »

À dire : cette API n'existe pas dans la version publiée d'apollo-angular (on l'a vérifié dans ses types). On obtient le même comportement en écrivant nous-mêmes un signal rempli par `Apollo.query`.

« Montrez-moi un `computed` et un `effect` . »

`frontend/src/app/app.ts`

```
badgeManagers = computed(() => {
  const w = this.gameService.world();
  return w ? w.managers.filter((m) => !m.unlocked && w.money >=
m.seuil).length : 0;
});

effect(() => {
  const msg = this.gameService.snackmessage();
  if (msg.texte) this.snackBar.open(msg.texte, 'ok', { duration: 2000 });
});
```

À dire : `computed` recalcule une valeur dès qu'un signal lu à l'intérieur change : ici le nombre de managers abordables, affiché dans le badge seulement s'il est supérieur à 0. `effect` exécute une action quand un signal change : ici, ouvrir le SnackBar à chaque nouveau message. Le message contient un compteur pour qu'un même texte deux fois de suite s'affiche quand même.

« Que se passe-t-il quand on clique sur Acheter ? »

```
async buyProduct(qt: number, product: Product) {
  const world = this.world();
  const cost = this.coutTotal(product, qt);
  if (qt <= 0 || world.money < cost) return;
  // 1. mise à jour immédiate de l'affichage
  this.world.set({ ...world, products: newproductlist, money: world.money -
cost });
  this.notify(`Bought ${qt} ${product.name}`);
  this.checkUnlocks(product.id);
  // 2. envoi au serveur
  await this.mutate(AcheterQtProduitDocument, { user: this.user(), id:
product.id, quantite: qt });
}
```

À dire : c'est une **mise à jour optimiste**. On ne modifie jamais l'objet existant : on crée un nouvel objet (`{ ...world, ... }`), sinon le signal ne verrait pas de changement. Puis la mutation part au serveur, qui refait le calcul de son côté. Si le serveur refuse, un message « Action rejected » s'affiche.

« Comment calculez-vous la quantité en mode Max ? »

```
let n = 0, coutUnitaire = product.cout, coutTotal = 0;
while (coutTotal + coutUnitaire <= money) {
  coutTotal += coutUnitaire;
  coutUnitaire *= product.croissance;
  n++;
}
return n;
```

À dire : on achète virtuellement un exemplaire de plus tant que le coût cumulé reste dans le budget. C'est la même suite de prix que côté serveur.

« Comment le composant produit reçoit-il ses données ? »

```
// produit.ts
prod = input<Product | undefined>();
```

```
<!-- app.html -->
@for (p of world.products; track p.id) {
  <app-produit [prod]="p" [qtmulti]="gameService.qtmulti()"></app-produit>
}
```

À dire : `input()` est la version signal de `@Input()` : le parent passe le produit, l'enfant le lit avec `this.prod()`. `@for` avec `track p.id` permet à Angular de réutiliser chaque carte au lieu de tout reconstruire quand le monde change.

« Comment anime-t-on la barre de production ? »

`frontend/src/app/produit/produit.ts`

```
const animate = (currentTime: number) => {
  const elapsed = currentTime - this.starttime;
  const progressPercent = Math.min((elapsed / p.vitesse) * 100, 100);
  this.progress.set(progressPercent);
  if (progressPercent < 100) {
    requestAnimationFrame(animate);
    return;
  }
  this.gameService.productionDone(p, 1);
};
requestAnimationFrame(animate);
```

À dire : `requestAnimationFrame` appelle la fonction à chaque image du navigateur (environ 60 par seconde). On calcule le pourcentage à partir du temps réel écoulé, pas en ajoutant un pas fixe : la barre reste juste même si le navigateur ralentit. À 100 %, on crédite le gain.

« Comment la production redémarre-t-elle seule avec un manager ? »

```
effect(() => {
  const p = this.prod();
  if (p?.managerUnlocked && !this.animating()) {
    untracked(() => this.demarrer(p));
  }
});
```

À dire : dès que le produit a un manager et ne tourne pas, l'`effect` lance la production. Dans la boucle d'animation, avec un manager, on crédite les cycles terminés et on continue au lieu de s'arrêter. `untracked` évite que les signaux lus pendant le démarrage relancent l'effet en boucle.

« Où est gardé le pseudo ? »

À dire : dans le `localStorage` du navigateur (`localStorage.setItem('username', ...)`), relu au démarrage par `initUsername()` . Sans pseudo enregistré, on en tire un au hasard (`Captain1234`) et on l'enregistre tout de suite, sinon chaque rechargement créerait un nouveau joueur.

Si une question vous bloque

Une réponse honnête et raisonnée vaut mieux qu'une réponse inventée. Quatre réflexes :

1. **Ouvrir le fichier et lire à voix haute.** « Je vous montre : cette méthode lit le monde, puis… ». Le code est sous vos yeux, le jury le sait.

2. **Remonter au principe.** Presque tout se ramène à une de ces trois idées : le serveur recalcule à partir du temps écoulé ; le client met à jour le signal `world` puis prévient le serveur ; le schéma GraphQL est le contrat commun.
3. **Suivre le chemin d'un clic.** Bouton dans `app.html` ou `produit.html` → méthode du `GameService` → mutation dans `queries.graphql` → resolver → `AppService` → sauvegarde JSON.
4. **Dire ce qu'on ferait pour vérifier.** « Je ne suis pas sûr, mais je le testerais en… » (un test Vitest, la console du navigateur, l'onglet Network).

Pour vous entraîner : l'un pose une question de cette section au hasard, l'autre répond sans lire, en ouvrant le fichier dans l'éditeur. Puis vous inversez.